RPG Game

Generated by Doxygen 1.9.1

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 Class Documentation 5

3.1 Action Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 Action() . . . 6

3.1.3 Member Function Documentation . . . 6

3.1.3.1 attempt() . . . 7

3.1.3.2 execute() . . . 7

3.1.3.3 getDesc() . . . 7

3.1.3.4 getManaUsage() . . . 8

3.1.3.5 getName() . . . 8

3.1.3.6 setDesc() . . . 8

3.1.3.7 setManaUsage() . . . 8

3.1.3.8 setName() . . . 9

3.2 Armour Class Reference . . . 9

3.2.1 Detailed Description . . . 10

3.2.2 Constructor & Destructor Documentation . . . 10

3.2.2.1 Armour() . . . 10

3.2.3 Member Function Documentation . . . 11

3.2.3.1 getWeight() . . . 11

3.2.3.2 setWeight() . . . 11

3.3 Bard Class Reference . . . 11

3.3.1 Detailed Description . . . 13

3.3.2 Constructor & Destructor Documentation . . . 13

3.3.2.1 Bard() . . . 13

3.3.3 Member Function Documentation . . . 14

3.3.3.1 clone() . . . 14

3.3.3.2 getLuck() . . . 14

3.3.3.3 setLuck() . . . 14

3.4 Battle Class Reference . . . 15

3.4.1 Detailed Description . . . 16

3.4.2 Member Function Documentation . . . 16

3.4.2.1 calculate_damage() . . . 16

3.4.2.2 get_turn() . . . 16

3.4.2.3 get_turn_block() . . . 16

3.4.2.4 getActionsCatalogue() . . . 17

3.4.2.5 getItemsCatalogue() . . . 17

3.4.2.6 getReportMessage() . . . 17
3.4.2.7 is_critical() . . . 17
3.4.2.8 is_enough_mana() . . . 18
3.4.2.9 set_turn() . . . 18
3.4.2.10 set_turn_block() . . . 18
3.4.2.11 setActionsCatalogue() . . . 19
3.4.2.12 setItemsCatalogue() . . . 19
3.4.2.13 setReportMessage() . . . 19
3.5 Character Class Reference . . . 19
3.5.1 Detailed Description . . . 22
3.5.2 Constructor & Destructor Documentation . . . 22
3.5.2.1 Character() . . . 22
3.5.3 Member Function Documentation . . . 22
3.5.3.1 getArmour() . . . 22
3.5.3.2 getAttackPower() . . . 23
3.5.3.3 getCid() . . . 23
3.5.3.4 getCritChance() . . . 23
3.5.3.5 getDefence() . . . 23
3.5.3.6 getHealingBonus() . . . 24
3.5.3.7 getHealingRate() . . . 24
3.5.3.8 getHealth() . . . 24
3.5.3.9 getIntelligence() . . . 24
3.5.3.10 getItem() . . . 25
3.5.3.11 getLuck() . . . 25
3.5.3.12 getMana() . . . 25
3.5.3.13 getName() . . . 25
3.5.3.14 getWeapon() . . . 26
3.5.3.15 setArmour() . . . 26
3.5.3.16 setAttackPower() . . . 26
3.5.3.17 setCritChance() . . . 26
3.5.3.18 setDefence() . . . 27
3.5.3.19 setHealingBonus() . . . 27
3.5.3.20 setHealingRate() . . . 27
3.5.3.21 setHealth() . . . 27
3.5.3.22 setIntelligence() . . . 28
3.5.3.23 setItem() . . . 28
3.5.3.24 setLuck() . . . 28
3.5.3.25 setMana() . . . 29
3.5.3.26 setName() . . . 29
3.5.3.27 setWeapon() . . . 29
3.6 Distract Class Reference . . . 30
3.6.1 Detailed Description . . . 31

3.6.2 Constructor & Destructor Documentation . . . 31
3.6.2.1 Distract() . . . 31
3.6.3 Member Function Documentation . . . 31
3.6.3.1 execute() . . . 31
3.7 Dodge Class Reference . . . 32
3.7.1 Detailed Description . . . 33
3.7.2 Constructor & Destructor Documentation . . . 33
3.7.2.1 Dodge() . . . 33
3.7.3 Member Function Documentation . . . 33
3.7.3.1 attempt() . . . 33
3.8 Druid Class Reference . . . 34
3.8.1 Detailed Description . . . 35
3.8.2 Constructor & Destructor Documentation . . . 35
3.8.2.1 Druid() . . . 35
3.8.3 Member Function Documentation . . . 36
3.8.3.1 clone() . . . 36
3.8.3.2 getHealingBonus() . . . 36
3.8.3.3 setHealingBonus() . . . 36
3.9 Enemy Class Reference . . . 37
3.9.1 Detailed Description . . . 38
3.9.2 Constructor & Destructor Documentation . . . 38
3.9.2.1 Enemy() . . . 38
3.9.3 Member Function Documentation . . . 39
3.9.3.1 getCritChance() . . . 39
3.9.3.2 getEquippingFlag() . . . 39
3.9.3.3 getNextAction() . . . 40
3.9.3.4 setCritChance() . . . 40
3.9.3.5 setEquippingFlag() . . . 41
3.9.3.6 setNextAction() . . . 41
3.10 EnemyAI Class Reference . . . 41
3.10.1 Detailed Description . . . 42
3.10.2 Member Function Documentation . . . 42
3.10.2.1 chooseAction() . . . 42
3.11 Equip Class Reference . . . 42
3.11.1 Detailed Description . . . 43
3.11.2 Constructor & Destructor Documentation . . . 43
3.11.2.1 Equip() . . . 43
3.11.3 Member Function Documentation . . . 44
3.11.3.1 execute() . . . 44
3.12 Game Class Reference . . . 44
3.12.1 Detailed Description . . . 46
3.12.2 Member Function Documentation . . . 46

3.12.2.1 getCharacters() . . . 47
3.12.2.2 getGeneralActions() . . . 47
3.12.2.3 getState() . . . 47
3.12.2.4 init_actions() . . . 47
3.12.2.5 init_items() . . . 48
3.12.2.6 is_numeric() . . . 48
3.13 Item Class Reference . . . 48
3.13.1 Detailed Description . . . 50
3.13.2 Constructor & Destructor Documentation . . . 51
3.13.2.1 Item() . . . 51
3.13.3 Member Function Documentation . . . 51
3.13.3.1 getAttackBonus() . . . 51
3.13.3.2 getCritChanceBonus() . . . 52
3.13.3.3 getDefenceBonus() . . . 52
3.13.3.4 getDesc() . . . 52
3.13.3.5 getDurability() . . . 52
3.13.3.6 getHBBonus() . . . 53
3.13.3.7 getHealthBonus() . . . 53
3.13.3.8 getIntBonus() . . . 53
3.13.3.9 getLuckBonus() . . . 53
3.13.3.10 getManaBonus() . . . 54
3.13.3.11 getName() . . . 54
3.13.3.12 getType() . . . 54
3.13.3.13 setAttackBonus() . . . 54
3.13.3.14 setCritChanceBonus() . . . 55
3.13.3.15 setDefenceBonus() . . . 55
3.13.3.16 setDesc() . . . 55
3.13.3.17 setDurability() . . . 55
3.13.3.18 setHBBonus() . . . 56
3.13.3.19 setHealthBonus() . . . 56
3.13.3.20 setIntBonus() . . . 56
3.13.3.21 setLuckBonus() . . . 57
3.13.3.22 setManaBonus() . . . 57
3.13.3.23 setName() . . . 57
3.14 Knight Class Reference . . . 57
3.14.1 Detailed Description . . . 59
3.14.2 Constructor & Destructor Documentation . . . 59
3.14.2.1 Knight() . . . 59
3.14.3 Member Function Documentation . . . 60
3.14.3.1 clone() . . . 60
3.14.3.2 getCritChance() . . . 60
3.14.3.3 setCritChance() . . . 60

3.15 Player Class Reference . . . 61
3.15.1 Detailed Description . . . 62
3.15.2 Constructor & Destructor Documentation . . . 62
3.15.2.1 Player() . . . 62
3.15.3 Member Function Documentation . . . 63
3.15.3.1 clone() . . . 63
3.15.3.2 getClassActions() . . . 63
3.15.3.3 getClassItems() . . . 64
3.15.3.4 getDesc() . . . 64
3.15.3.5 getItem() . . . 64
3.15.3.6 setClassActions() . . . 64
3.15.3.7 setClassItems() . . . 65
3.15.3.8 setDesc() . . . 65
3.15.3.9 setItem() . . . 65
3.16 Spell Class Reference . . . 66
3.16.1 Detailed Description . . . 66
3.16.2 Constructor & Destructor Documentation . . . 67
3.16.2.1 Spell() . . . 67
3.16.3 Member Function Documentation . . . 67
3.16.3.1 exucute() . . . 67
3.17 Statistics Class Reference . . . 68
3.17.1 Detailed Description . . . 68
3.17.2 Member Function Documentation . . . 68
3.17.2.1 get_games() . . . 68
3.17.2.2 get_score() . . . 68
3.17.2.3 set_games() . . . 68
3.17.2.4 set_score() . . . 69
3.18 StatModifyingAction Class Reference . . . 69
3.18.1 Detailed Description . . . 70
3.18.2 Constructor & Destructor Documentation . . . 70
3.18.2.1 StatModifyingAction() . . . 71
3.18.3 Member Function Documentation . . . 71
3.18.3.1 execute() . . . 71
3.18.3.2 getModifiedStat() . . . 71
3.18.3.3 getQuantity() . . . 72
3.18.3.4 setModifiedStat() . . . 72
3.18.3.5 setQuantity() . . . 72
3.19 Weapon Class Reference . . . 73
3.19.1 Detailed Description . . . 74
3.19.2 Constructor & Destructor Documentation . . . 74
3.19.2.1 Weapon() . . . 74
3.19.3 Member Function Documentation . . . 74

3.19.3.1 getLevel() 74
3.19.3.2 setLevel() 75
3.20 Wizard Class Reference 75
3.20.1 Detailed Description 76
3.20.2 Constructor & Destructor Documentation 76
3.20.2.1 Wizard() 77
3.20.3 Member Function Documentation 77
3.20.3.1 clone() 77
3.20.3.2 getIntelligence() 78
3.20.3.3 setIntelligence() 78

Index **79**

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Action . . . 5
  - Distract . . . 30
  - Dodge . . . 32
  - Equip . . . 42
  - Spell . . . 66
  - StatModifyingAction . . . 69
- Battle . . . 15
- Character . . . 19
  - Enemy . . . 37
  - Player . . . 61
    - Bard . . . 11
    - Druid . . . 34
    - Knight . . . 57
    - Wizard . . . 75
- EnemyAI . . . 41
- Game . . . 44
- Item . . . 48
  - Armour . . . 9
  - Weapon . . . 73
- Statistics . . . 68

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Action
: Represents a generic action in the game with a name, description, and mana cost 5

Armour
: Represents a piece of armour in the game, providing defense and weight 9

Bard
: Player class representing a Bard character with a luck attribute 11

Battle
: Manages battle state, actions, items, and calculates damage in the game 15

Character
: Represents a generic character in the game with stats and equipment 19

Distract
: Action that causes the opponent to skip their next turn 30

Dodge
: Action that attempts to dodge an opponent's attack with a probabilistic success 32

Druid
: Player class representing a Druid with healing bonus ability 34

Enemy
: Represents an enemy character in the game with AI-specific attributes 37

EnemyAI
: Implements logic for enemy action decision-making during battle 41

Equip
: Action to equip an item, weapon, or armour on a character 42

Game
: Main game class controlling state, characters, battles, and user interaction 44

Item
: Represents an item that characters can equip or use, with various stat bonuses 48

Knight
: Player class representing a Knight character with critical chance attribute 57

Player
: Represents a player character, inheriting from Character 61

Spell
: Represents a spell action with effect based on intelligence and attack power 66

Statistics
: Keeps track of the player's score and number of games played 68

StatModifyingAction
: Action that modifies character stats such as health, mana, attack, or defense 69

Weapon
Represents a weapon item with additional level attribute 73
Wizard
Player class representing a Wizard with intelligence attribute 75

Chapter 3

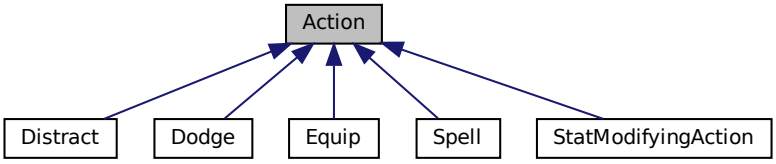
Class Documentation

3.1 Action Class Reference

Represents a generic action in the game with a name, description, and mana cost.

`#include <Action.h>`

Inheritance diagram for Action:



Public Member Functions

- Action (const std::string &name, const std::string &desc, int mana)

  Constructs an Action.
- virtual ~Action ()

  Destructor.
- void setName (const std::string &name)

  Sets the name of the action.
- const std::string & getName () const

  Gets the action name.
- void setDesc (const std::string &desc)

  Sets the description for the action.
- const std::string & getDesc ()

  Gets the action description.
- void setManaUsage (int mana)

  Sets the mana cost for the action.

- int getManaUsage ()

  Gets the mana cost to perform the action.
- virtual void execute (Character &, Character &, Battle &, std::shared_ptr< Item >, Character &)

  Executes the action.
- virtual bool attempt (Character &)

  Attempts to perform the action, may include success probability.

Protected Attributes

- std::string name

  Action name.
- std::string description

  Description of what the action does.
- int mana_usage

  Mana points required to perform the action.

3.1.1 Detailed Description

Represents a generic action in the game with a name, description, and mana cost.

Base class for specific actions that characters can perform during battle.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Action()

```
Action::Action (
            const std::string & name,
            const std::string & desc,
            int mana )
```

Constructs an Action.

Parameters

name	Name of the action.
desc	Description of the action.
mana	Mana cost to perform the action.

3.1.3 Member Function Documentation

3.1.3.1 attempt()

```
bool Action::attempt (
            Character &  )  [virtual]
```

Attempts to perform the action, may include success probability.

Parameters

performer	Character attempting the action.

Returns

True on success, false otherwise.

Reimplemented in Dodge.

3.1.3.2 execute()

```
void Action::execute (
            Character & ,
            Character & ,
            Battle & ,
            std::shared_ptr< Item > ,
            Character &  )  [virtual]
```

Executes the action.

Parameters

performer	Character performing the action.
target	Character targeted by the action.
battle	Current battle context.
item	Optional item used during the action.
item_target	Character receiving the item effect.

Reimplemented in StatModifyingAction, Distract, and Equip.

3.1.3.3 getDesc()

```
const std::string & Action::getDesc ( )
```

Gets the action description.

Returns

Reference to the description string.

3.1.3.4 getManaUsage()

```
int Action::getManaUsage ( )
```

Gets the mana cost to perform the action.

Returns

Integer mana usage.

3.1.3.5 getName()

```
const std::string & Action::getName ( ) const
```

Gets the action name.

Returns

Reference to the action's name.

3.1.3.6 setDesc()

```
void Action::setDesc (
            const std::string & desc )
```

Sets the description for the action.

Parameters

desc	New description.

3.1.3.7 setManaUsage()

```
void Action::setManaUsage (
            int mana )
```

Sets the mana cost for the action.

Parameters

mana	Mana usage to set.

3.1.3.8 setName()

```
void Action::setName (
            const std::string & name )
```

Sets the name of the action.

Parameters

name	New name to set.

The documentation for this class was generated from the following files:

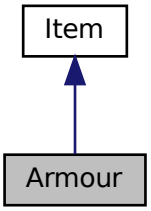
- source/Action.h
- source/Action.cpp

3.2 Armour Class Reference

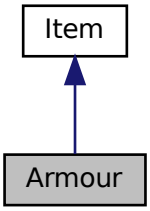
Represents a piece of armour in the game, providing defense and weight.

```
#include <Armour.h>
```

Inheritance diagram for Armour:



Collaboration diagram for Armour:



Public Member Functions

- Armour (const std::string &name, const std::string &desc, int durability, int defence_bonus, int attack_bonus, int weight)

  Constructs an Armour object.
- void setWeight (int weight)

  Sets the weight of the armour.
- int getWeight () const

  Gets the weight of the armour.

Protected Attributes

- int weight

  Weight of the armour.

3.2.1 Detailed Description

Represents a piece of armour in the game, providing defense and weight.

Inherits from Item and adds a weight attribute.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Armour()

```
Armour::Armour (
            const std::string & name,
            const std::string & desc,
            int durability,
            int defence_bonus,
            int attack_bonus,
            int weight )
```

Constructs an Armour object.

Parameters

name	Name of the armour piece.
desc	Description of the armour.
durability	Durability value.
defence_bonus	Defense bonus provided by the armour.
attack_bonus	Attack bonus provided by the armour.
weight	Weight of the armour.

3.2.3 Member Function Documentation

3.2.3.1 getWeight()

```
int Armour::getWeight ( ) const
```

Gets the weight of the armour.

Returns

Weight value.

3.2.3.2 setWeight()

```
void Armour::setWeight (
            int weight )
```

Sets the weight of the armour.

Parameters

weight	New weight value.

The documentation for this class was generated from the following files:

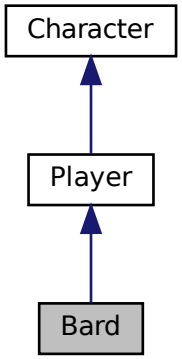
- source/Armour.h
- source/Armour.cpp

3.3 Bard Class Reference

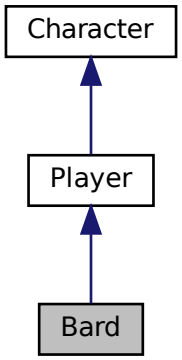
Player class representing a Bard character with a luck attribute.

`#include <Bard.h>`

Inheritance diagram for Bard:



Collaboration diagram for Bard:



Public Member Functions

- Bard (const std::string &name, int health, int mana, int attack_power, int defence, int hr, const std::string &desc, std::shared_ptr< Item > item, double luck)

  Constructs a Bard with specific attributes.
- Bard ()

  Default constructor for Bard with initial standard stats.
- ~Bard ()

  Destructor.
- void setLuck (double luck) override

Sets luck value.

- double getLuck () const override

  Gets the current luck value.

- std::shared_ptr< Player > clone () const override

  Creates a polymorphic clone of the Bard object.

Protected Attributes

- double luck

  Luck factor affecting character abilities.

3.3.1 Detailed Description

Player class representing a Bard character with a luck attribute.

Inherits from Player and adds luck functionality.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Bard()

```
Bard::Bard (
            const std::string & name,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            const std::string & desc,
            std::shared_ptr< Item > item,
            double luck )
```

Constructs a Bard with specific attributes.

Parameters

name	Character name.
health	Health points.
mana	Mana points.
attack_power	Attack power.
defence	Defense value.
hr	Healing rate.
desc	Description text.
item	Starting item.
luck	Luck factor affecting probabilities.

3.3.3 Member Function Documentation

3.3.3.1 clone()

```
std::shared_ptr< Player > Bard::clone ( ) const  [override], [virtual]
```

Creates a polymorphic clone of the Bard object.

Returns

Shared pointer to a copy of this Bard.

Reimplemented from Player.

3.3.3.2 getLuck()

```
double Bard::getLuck ( ) const  [override], [virtual]
```

Gets the current luck value.

Returns

Luck value.

Reimplemented from Character.

3.3.3.3 setLuck()

```
void Bard::setLuck (
            double luck )  [override], [virtual]
```

Sets luck value.

Parameters

luck	Luck value to set.

Reimplemented from Character.

The documentation for this class was generated from the following files:

- source/Bard.h
- source/Bard.cpp

3.4 Battle Class Reference

Manages battle state, actions, items, and calculates damage in the game.

`#include <Battle.h>`

Public Member Functions

- Battle ()
  Constructs a Battle with default settings.
- void set_turn (bool turn)
  Sets the current turn indicator.
- bool get_turn () const
  Gets the current turn indicator.
- void set_turn_block (int n)
  Sets the count of turn blocks (to skip turns).
- int get_turn_block ()
  Gets the current number of turn blocks.
- void setReportMessage (int m)
  Sets the report message code.
- int getReportMessage ()
  Gets the current report message code.
- int calculate_damage (Character &performer, Character &target)
  Calculates damage from performer to target based on stats and items.
- bool is_critical (Character &performer)
  Determines whether the attack is critical based on chance.
- bool is_enough_mana (Character &performer, Action &action)
  Checks if performer has enough mana for the given action.
- void setActionsCatalogue (std::vector< std::shared_ptr< Action >> a)
  Sets the list of available actions in this battle.
- const std::vector< std::shared_ptr< Action > > & getActionsCatalogue ()
  Gets the list of available actions in this battle.
- void setItemsCatalogue (std::vector< std::shared_ptr< Item >> i)
  Sets the list of available items in this battle.
- const std::vector< std::shared_ptr< Item > > & getItemsCatalogue ()
  Gets the list of available items in this battle.

Protected Attributes

- bool turn
  True if it is the player's turn.
- int turn_block
  Number of remaining turns blocked.
- int report_message
  Current report message index.
- std::vector< std::shared_ptr< Action > > actions_catalogue
  Available actions.
- std::vector< std::shared_ptr< Item > > items_catalogue
  Available items.

3.4.1 Detailed Description

Manages battle state, actions, items, and calculates damage in the game.

3.4.2 Member Function Documentation

3.4.2.1 calculate_damage()

```
int Battle::calculate_damage (
            Character & performer,
            Character & target )
```

Calculates damage from performer to target based on stats and items.

Parameters

performer	Character performing the attack.
target	Character targeted.

Returns

Calculated damage as integer.

3.4.2.2 get_turn()

```
bool Battle::get_turn ( ) const
```

Gets the current turn indicator.

Returns

True if player's turn, false otherwise.

3.4.2.3 get_turn_block()

```
int Battle::get_turn_block ( )
```

Gets the current number of turn blocks.

Returns

Integer turn block count.

3.4.2.4 getActionsCatalogue()

```
const std::vector< std::shared_ptr< Action > > & Battle::getActionsCatalogue ( )
```

Gets the list of available actions in this battle.

Returns

Const reference to vector of shared pointers to actions.

3.4.2.5 getItemsCatalogue()

```
const std::vector< std::shared_ptr< Item > > & Battle::getItemsCatalogue ( )
```

Gets the list of available items in this battle.

Returns

Const reference to vector of shared pointers to items.

3.4.2.6 getReportMessage()

```
int Battle::getReportMessage ( )
```

Gets the current report message code.

Returns

Message code index.

3.4.2.7 is_critical()

```
bool Battle::is_critical (
            Character & performer )
```

Determines whether the attack is critical based on chance.

Parameters

performer	Character performing the attack.

Returns

True if critical hit, false otherwise.

3.4.2.8 is_enough_mana()

```
bool Battle::is_enough_mana (
            Character & performer,
            Action & action )
```

Checks if performer has enough mana for the given action.

Parameters

performer	Character performing the action.
action	Action to be performed.

Returns

True if enough mana, false otherwise.

3.4.2.9 set_turn()

```
void Battle::set_turn (
            bool turn )
```

Sets the current turn indicator.

Parameters

turn	True if it is player's turn, false otherwise.

3.4.2.10 set_turn_block()

```
void Battle::set_turn_block (
            int n )
```

Sets the count of turn blocks (to skip turns).

Parameters

n	Number of turn blocks.

3.4.2.11 setActionsCatalogue()

```
void Battle::setActionsCatalogue (
            std::vector< std::shared_ptr< Action >> a )
```

Sets the list of available actions in this battle.

Parameters

a	Vector of shared pointers to actions.

3.4.2.12 setItemsCatalogue()

```
void Battle::setItemsCatalogue (
            std::vector< std::shared_ptr< Item >> i )
```

Sets the list of available items in this battle.

Parameters

i	Vector of shared pointers to items.

3.4.2.13 setReportMessage()

```
void Battle::setReportMessage (
            int m )
```

Sets the report message code.

Parameters

m	Message code index.

The documentation for this class was generated from the following files:

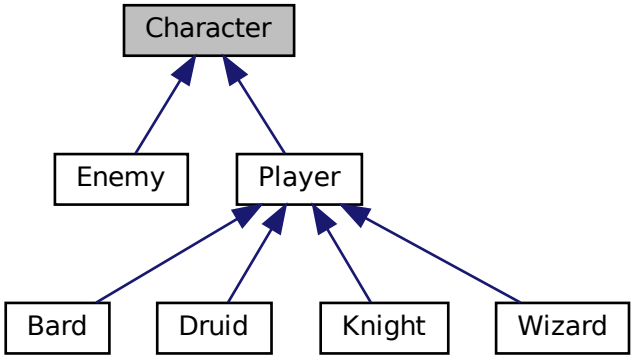
- source/Battle.h
- source/Battle.cpp

3.5 Character Class Reference

Represents a generic character in the game with stats and equipment.

`#include <Character.h>`

Inheritance diagram for Character:



Public Member Functions

- Character (const std::string &name, int cid, int health, int mana, int attack_power, int defence, int hr)
  Constructs a Character with given attributes.
- virtual ~Character ()
  Destructor.
- void setName (const std::string &n)
  Sets the character's name.
- std::string getName () const
  Gets the character's name.
- void setHealth (int h)
  Sets the character's health.
- int getHealth () const
  Gets the character's health.
- void setMana (int m)
  Sets the character's mana.
- int getMana () const
  Gets the character's mana.
- void setAttackPower (int ap)
  Sets the attack power.
- int getAttackPower () const
  Gets the attack power.
- void setDefence (int d)
  Sets the defense rating.
- int getDefence () const
  Gets the defense rating.
- void setHealingRate (int hr)
  Sets the healing rate.
- int getHealingRate ()

Gets the healing rate.

- virtual void setLuck (double)

  Sets the luck stat (default empty in base).
- virtual double getLuck () const

  Gets the luck stat (default zero in base).
- virtual void setHealingBonus (int)

  Sets the healing bonus (default empty).
- virtual int getHealingBonus () const

  Gets the healing bonus (default zero).
- virtual void setCritChance (double)

  Sets the critical chance (default empty).
- virtual double getCritChance () const

  Gets the critical chance (default zero).
- virtual void setIntelligence (int)

  Sets the intelligence stat (default empty).
- virtual int getIntelligence () const

  Gets the intelligence stat (default zero).
- void setArmour (std::shared_ptr< Armour > a)

  Sets the armour equipped.
- std::shared_ptr< Armour > getArmour () const

  Gets the currently equipped armour.
- void setWeapon (std::shared_ptr< Weapon > w)

  Sets the weapon equipped.
- std::shared_ptr< Weapon > getWeapon () const

  Gets the currently equipped weapon.
- virtual void setItem (std::shared_ptr< Item >)

  Sets the currently held item (default empty).
- virtual std::shared_ptr< Item > getItem () const

  Gets the currently held item (default nullptr).
- int getCid ()

  Gets the class ID.

Protected Attributes

- std::string name

  Character's name.
- int class_id

  Class identifier.
- int health

  Health points.
- int mana

  Mana points.
- int attack_power

  Attack power.
- int defence

  Defense value.
- int healing_rate

  Healing rate.
- std::shared_ptr< Weapon > weapon

  Equipped weapon.
- std::shared_ptr< Armour > armour

  Equipped armour.

3.5.1 Detailed Description

Represents a generic character in the game with stats and equipment.

3.5.2 Constructor & Destructor Documentation

3.5.2.1 Character()

```
Character::Character (
            const std::string & name,
            int cid,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr )
```

Constructs a Character with given attributes.

Parameters

name	Character's name.
cid	Class ID to identify character class.
health	Health points.
mana	Mana points.
attack_power	Attack power.
defence	Defense rating.
hr	Healing rate.

3.5.3 Member Function Documentation

3.5.3.1 getArmour()

```
std::shared_ptr< Armour > Character::getArmour ( ) const
```

Gets the currently equipped armour.

Returns

Shared pointer to Armour.

3.5.3.2 getAttackPower()

```
int Character::getAttackPower ( ) const
```

Gets the attack power.

Returns

Attack power value.

3.5.3.3 getCid()

```
int Character::getCid ( )
```

Gets the class ID.

Returns

Class ID integer.

3.5.3.4 getCritChance()

```
double Character::getCritChance ( ) const  [virtual]
```

Gets the critical chance (default zero).

Returns

Critical chance value.

Reimplemented in Knight, and Enemy.

3.5.3.5 getDefence()

```
int Character::getDefence ( ) const
```

Gets the defense rating.

Returns

Defense value.

3.5.3.6 getHealingBonus()

```
int Character::getHealingBonus ( ) const  [virtual]
```

Gets the healing bonus (default zero).

Returns

Healing bonus.

Reimplemented in Druid.

3.5.3.7 getHealingRate()

```
int Character::getHealingRate ( )
```

Gets the healing rate.

Returns

Healing rate value.

3.5.3.8 getHealth()

```
int Character::getHealth ( ) const
```

Gets the character's health.

Returns

Current health points.

3.5.3.9 getIntelligence()

```
int Character::getIntelligence ( ) const  [virtual]
```

Gets the intelligence stat (default zero).

Returns

Intelligence value.

Reimplemented in Wizard.

3.5.3.10 getItem()

```
std::shared_ptr< Item > Character::getItem ( ) const  [virtual]
```

Gets the currently held item (default nullptr).

Returns

Shared pointer to Item.

Reimplemented in Player.

3.5.3.11 getLuck()

```
double Character::getLuck ( ) const  [virtual]
```

Gets the luck stat (default zero in base).

Returns

Luck value.

Reimplemented in Bard.

3.5.3.12 getMana()

```
int Character::getMana ( ) const
```

Gets the character's mana.

Returns

Current mana points.

3.5.3.13 getName()

```
std::string Character::getName ( ) const
```

Gets the character's name.

Returns

Name as string.

3.5.3.14 getWeapon()

```
std::shared_ptr< Weapon > Character::getWeapon ( ) const
```

Gets the currently equipped weapon.

Returns

Shared pointer to Weapon.

3.5.3.15 setArmour()

```
void Character::setArmour (
            std::shared_ptr< Armour > a )
```

Sets the armour equipped.

Parameters

a	Shared pointer to Armour.

3.5.3.16 setAttackPower()

```
void Character::setAttackPower (
            int ap )
```

Sets the attack power.

Parameters

ap	Attack power value.

3.5.3.17 setCritChance()

```
void Character::setCritChance (
            double ) [virtual]
```

Sets the critical chance (default empty).

Parameters

Not	used.

Reimplemented in Knight, and Enemy.

3.5.3.18 setDefence()

```
void Character::setDefence (
            int d )
```

Sets the defense rating.

Parameters

d	Defense value.

3.5.3.19 setHealingBonus()

```
void Character::setHealingBonus (
            int  )  [virtual]
```

Sets the healing bonus (default empty).

Parameters

Not	used.

Reimplemented in Druid.

3.5.3.20 setHealingRate()

```
void Character::setHealingRate (
            int hr )
```

Sets the healing rate.

Parameters

hr	Healing rate value.

3.5.3.21 setHealth()

```
void Character::setHealth (
```

```
            int h )
```

Sets the character's health.

Parameters

h	Health points.

3.5.3.22 setIntelligence()

```
void Character::setIntelligence (
            int  )  [virtual]
```

Sets the intelligence stat (default empty).

Parameters

Not	used.

Reimplemented in Wizard.

3.5.3.23 setItem()

```
void Character::setItem (
            std::shared_ptr< Item >  )  [virtual]
```

Sets the currently held item (default empty).

Parameters

Shared	pointer to Item.

Reimplemented in Player.

3.5.3.24 setLuck()

```
void Character::setLuck (
            double  )  [virtual]
```

Sets the luck stat (default empty in base).

Parameters

Not	used.

Reimplemented in Bard.

3.5.3.25 setMana()

```
void Character::setMana (
            int m )
```

Sets the character's mana.

Parameters

m	Mana points.

3.5.3.26 setName()

```
void Character::setName (
            const std::string & n )
```

Sets the character's name.

Parameters

n	New name.

3.5.3.27 setWeapon()

```
void Character::setWeapon (
            std::shared_ptr< Weapon > w )
```

Sets the weapon equipped.

Parameters

w	Shared pointer to Weapon.

The documentation for this class was generated from the following files:

- source/Character.h

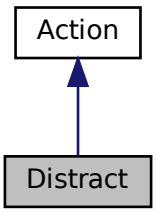
- source/Character.cpp

3.6 Distract Class Reference

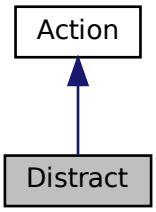
Action that causes the opponent to skip their next turn.

`#include <Distract.h>`

Inheritance diagram for Distract:



Collaboration diagram for Distract:



Public Member Functions

- Distract (const std::string &name, const std::string &desc, int mana)

  Constructs a Distract action.
- void execute (Character &performer, Character &, Battle &battle, std::shared_ptr< Item >, Character &) override

  Executes the distraction, setting the turn block in the battle.

Additional Inherited Members

3.6.1 Detailed Description

Action that causes the opponent to skip their next turn.

Derived from Action class.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 Distract()

```
Distract::Distract (
            const std::string & name,
            const std::string & desc,
            int mana )
```

Constructs a Distract action.

Parameters

name	Action name.
desc	Action description.
mana	Mana cost.

3.6.3 Member Function Documentation

3.6.3.1 execute()

```
void Distract::execute (
            Character & performer,
            Character & ,
            Battle & battle,
            std::shared_ptr< Item > ,
            Character & ) [override], [virtual]
```

Executes the distraction, setting the turn block in the battle.

Parameters

performer	Character performing the action.
unused	Target character (not used).
battle	Reference to the battle.
unused	Item used (not used).
unused	Character item target (not used).

Reimplemented from Action.

The documentation for this class was generated from the following files:

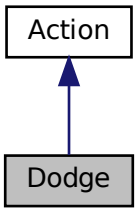
- source/Distract.h
- source/Distract.cpp

3.7 Dodge Class Reference

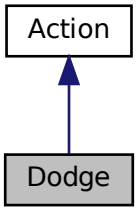
Action that attempts to dodge an opponent's attack with a probabilistic success.

`#include <Dodge.h>`

Inheritance diagram for Dodge:



Collaboration diagram for Dodge:



Public Member Functions

- Dodge (const std::string &name, const std::string &desc, int mana)

  Constructs a Dodge action.
- bool attempt (Character &performer) override

  Attempts to perform the dodge action.

Additional Inherited Members

3.7.1 Detailed Description

Action that attempts to dodge an opponent's attack with a probabilistic success.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Dodge()

```
Dodge::Dodge (
            const std::string & name,
            const std::string & desc,
            int mana )
```

Constructs a Dodge action.

Parameters

name	Action name.
desc	Description of action.
mana	Mana cost.

3.7.3 Member Function Documentation

3.7.3.1 attempt()

```
bool Dodge::attempt (
            Character & performer )  [override], [virtual]
```

Attempts to perform the dodge action.

Parameters

performer	Character attempting the dodge.

Returns

True if dodge is successful, false otherwise.

Reimplemented from Action.

The documentation for this class was generated from the following files:

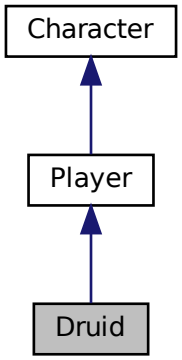
- source/Dodge.h
- source/Dodge.cpp

3.8 Druid Class Reference

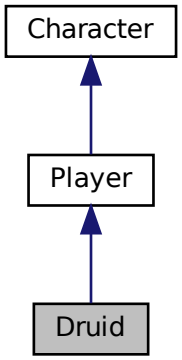
Player class representing a Druid with healing bonus ability.

`#include <Druid.h>`

Inheritance diagram for Druid:



Collaboration diagram for Druid:



Public Member Functions

- Druid (const std::string &name, int health, int mana, int attack_power, int defence, int hr, const std::string &desc, std::shared_ptr< Item > item, int healing_bonus)

  Constructs a Druid with specified attributes and healing bonus.
- Druid ()

  Default constructor initializing standard Druid stats.
- ~Druid ()

  Destructor.
- void setHealingBonus (int bonus) override

  Sets the healing bonus.
- int getHealingBonus () const override

  Gets the healing bonus.
- std::shared_ptr< Player > clone () const override

  Clones the Druid object polymorphically.

Protected Attributes

- int healing_bonus

  Additional healing bonus of the Druid.

3.8.1 Detailed Description

Player class representing a Druid with healing bonus ability.

Inherits from Player and adds healing bonus.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 Druid()

```
Druid::Druid (
            const std::string & name,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            const std::string & desc,
            std::shared_ptr< Item > item,
            int healing_bonus )
```

Constructs a Druid with specified attributes and healing bonus.

Parameters

name	Character name.
health	Health points.

Parameters

mana	Mana points.
attack_power	Attack power.
defence	Defense rating.
hr	Healing rate.
desc	Description.
item	Starting item.
healing_bonus	Additional healing bonus.

3.8.3 Member Function Documentation

3.8.3.1 clone()

```
std::shared_ptr< Player > Druid::clone ( ) const  [override], [virtual]
```

Clones the Druid object polymorphically.

Returns

Shared pointer to copied Druid.

Reimplemented from Player.

3.8.3.2 getHealingBonus()

```
int Druid::getHealingBonus ( ) const  [override], [virtual]
```

Gets the healing bonus.

Returns

Healing bonus value.

Reimplemented from Character.

3.8.3.3 setHealingBonus()

```
void Druid::setHealingBonus (
            int bonus )  [override], [virtual]
```

Sets the healing bonus.

Parameters

bonus	Healing bonus value.

Reimplemented from Character.

The documentation for this class was generated from the following files:

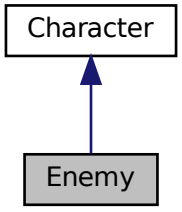
- source/Druid.h
- source/Druid.cpp

3.9 Enemy Class Reference

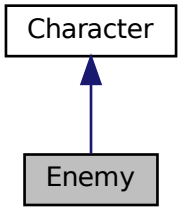
Represents an enemy character in the game with AI-specific attributes.

`#include <Enemy.h>`

Inheritance diagram for Enemy:



Collaboration diagram for Enemy:



Public Member Functions

- Enemy ()

  Default constructor for Enemy with initial standard stats.
- Enemy (const std::string &name, int health, int mana, int attack_power, int defence, int hr, double crit_chance, int equipping_flag, std::shared_ptr< Action > nxt_act)

  Constructs an Enemy with specific attributes.
- ~Enemy ()

  Destructor.
- void setCritChance (double chance)

  Sets the critical chance for the enemy.
- double getCritChance () const

  Gets the critical chance for the enemy.
- void setEquippingFlag (int f)

  Sets the equipping flag indicating item equip status.
- int getEquippingFlag ()

  Gets the equipping flag.
- void setNextAction (std::shared_ptr< Action > act)

  Sets the next planned action for the enemy.
- std::shared_ptr< Action > getNextAction () const

  Gets the next planned action.

Protected Attributes

- double crit_chance

  Enemy critical hit chance.
- int equipping_flag

  Enemy equip flag.
- std::shared_ptr< Action > next_action

  Enemy's next planned action.

3.9.1 Detailed Description

Represents an enemy character in the game with AI-specific attributes.

Inherits from Character with additional attributes like critical chance, equipping flags, and next planned action.

3.9.2 Constructor & Destructor Documentation

3.9.2.1 Enemy()

```
Enemy::Enemy (
            const std::string & name,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            double crit_chance,
            int equipping_flag,
            std::shared_ptr< Action > nxt_act )
```

Constructs an Enemy with specific attributes.

Parameters

name	Name of the enemy.
health	Health points.
mana	Mana points.
attack_power	Attack power.
defence	Defense rating.
hr	Healing rate.
crit_chance	Probability of critical hit.
equipping_flag	Flag indicating item equipping status.
nxt_act	Next planned action for the enemy.

3.9.3 Member Function Documentation

3.9.3.1 getCritChance()

```
double Enemy::getCritChance ( ) const  [virtual]
```

Gets the critical chance for the enemy.

Returns

Critical hit probability.

Reimplemented from Character.

3.9.3.2 getEquippingFlag()

```
int Enemy::getEquippingFlag ( )
```

Gets the equipping flag.

Returns

Equipping flag value.

3.9.3.3 getNextAction()

```
std::shared_ptr< Action > Enemy::getNextAction ( ) const
```

Gets the next planned action.

Returns

Shared pointer to next action.

3.9.3.4 setCritChance()

```
void Enemy::setCritChance (
            double chance )  [virtual]
```

Sets the critical chance for the enemy.

Parameters

chance	Critical hit probability.

Reimplemented from Character.

3.9.3.5 setEquippingFlag()

```
void Enemy::setEquippingFlag (
            int f )
```

Sets the equipping flag indicating item equip status.

Parameters

f	Flag value.

3.9.3.6 setNextAction()

```
void Enemy::setNextAction (
            std::shared_ptr< Action > act )
```

Sets the next planned action for the enemy.

Parameters

act	Shared pointer to the action.

The documentation for this class was generated from the following files:

- source/Enemy.h
- source/Enemy.cpp

3.10 EnemyAI Class Reference

Implements logic for enemy action decision-making during battle.

`#include <EnemyAI.h>`

Public Member Functions

- EnemyAI ()

  Default constructor.
- int chooseAction (Player &player, Enemy &enemy, Battle &battle)

  Chooses the next action for the enemy given player, enemy, and battle state.

3.10.1 Detailed Description

Implements logic for enemy action decision-making during battle.

3.10.2 Member Function Documentation

3.10.2.1 chooseAction()

```
int EnemyAI::chooseAction (
            Player & player,
            Enemy & enemy,
            Battle & battle )
```

Chooses the next action for the enemy given player, enemy, and battle state.

Parameters

player	The player character.
enemy	The enemy character.
battle	Current battle context.

Returns

Index code representing the chosen action.

The documentation for this class was generated from the following files:

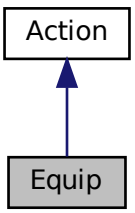
- source/EnemyAI.h
- source/EnemyAI.cpp

3.11 Equip Class Reference

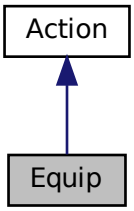
Action to equip an item, weapon, or armour on a character.

`#include <Equip.h>`

Inheritance diagram for Equip:



Collaboration diagram for Equip:



Public Member Functions

- Equip (const std::string &name, const std::string &description, int mana)

  Constructs an Equip action.
- void execute (Character &performer, Character &, Battle &, std::shared_ptr< Item > item, Character &item↩ _target) override

  Executes the equip action on the target character.

Additional Inherited Members

3.11.1 Detailed Description

Action to equip an item, weapon, or armour on a character.

Derived from Action.

3.11.2 Constructor & Destructor Documentation

3.11.2.1 Equip()

```
Equip::Equip (
            const std::string & name,
            const std::string & description,
            int mana )
```

Constructs an Equip action.

Parameters

name	Name of the action.
description	Description of the action.
mana	Mana cost (usually 0).

3.11.3 Member Function Documentation

3.11.3.1 execute()

```
void Equip::execute (
            Character & performer,
            Character & ,
            Battle & ,
            std::shared_ptr< Item > item,
            Character & item_target )  [override], [virtual]
```

Executes the equip action on the target character.

Parameters

performer	Character performing equip.
unused	Unused target character (not used).
battle	Current battle context.
item	Item to equip.
item_target	Character receiving the item.

Reimplemented from Action.

The documentation for this class was generated from the following files:

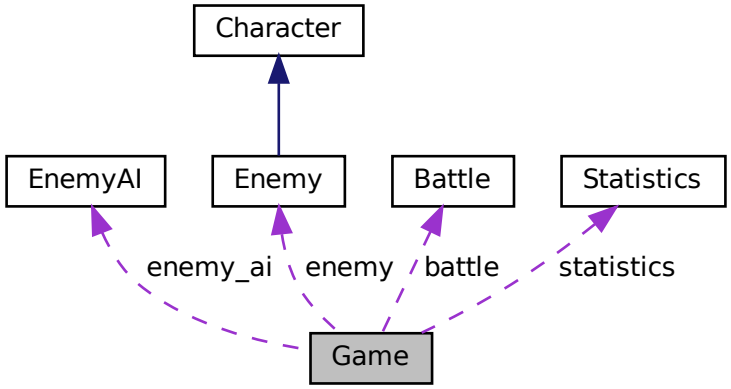
- source/Equip.h
- source/Equip.cpp

3.12 Game Class Reference

Main game class controlling state, characters, battles, and user interaction.

```
#include <Game.h>
```

Collaboration diagram for Game:



Public Member Functions

- Game ()

  Default constructor initializes game state.
- void start ()

  Starts the main game loop.
- void quit ()

  Quits the game and stops the game loop.
- void start_battle ()

  Starts a battle session.
- void edit_character ()

  Puts game into character edit mode.
- void settings ()

  Opens settings menu.
- void leaderboard ()

  Displays leaderboard or game statistics.
- void display ()

  Displays the current screen based on game state.
- void init_characters ()

  Initializes the available player characters.
- void init_items (std::shared_ptr< Player > bard, std::shared_ptr< Player > druid, std::shared_ptr< Player > knight, std::shared_ptr< Player > wizard)

  Initializes items associated with player classes and enemies.
- void init_actions (std::shared_ptr< Player > bard, std::shared_ptr< Player > druid, std::shared_ptr< Player > knight, std::shared_ptr< Player > wizard)

  Initializes actions available to player classes and general actions.
- bool is_numeric (std::string &input)

  Checks if a string input is numeric.
- void exit_battle ()

  Exits from battle and resets battle-related state.

- bool getState () const

  Gets the current state indicating if the game is running.
- const std::vector< std::shared_ptr< Player > > & getCharacters () const

  Gets the list of all player characters available in the game.
- const std::vector< std::shared_ptr< Action > > & getGeneralActions () const

  Gets the general actions available in the game.

Protected Attributes

- Battle battle

  Battle session instance.
- std::shared_ptr< Player > player

  Current player character.
- Enemy enemy

  Current enemy character.
- Statistics statistics

  Game statistics tracking.
- EnemyAI enemy_ai

  Enemy AI logic handler.
- int state

  Current game state indicator.
- int difficulty

  Difficulty setting.
- bool is_running

  Flag to keep game loop running.
- std::vector< std::shared_ptr< Player > > characters

  All player character options.
- std::vector< std::shared_ptr< Item > > general_items

  Items available generally.
- std::vector< std::shared_ptr< Item > > enemy_items

  Items available to enemy.
- std::vector< std::shared_ptr< Action > > general_actions

  General actions available.

3.12.1 Detailed Description

Main game class controlling state, characters, battles, and user interaction.

3.12.2 Member Function Documentation

3.12.2.1 getCharacters()

```
const std::vector< std::shared_ptr< Player > > & Game::getCharacters ( ) const
```

Gets the list of all player characters available in the game.

Returns

Const reference to a vector of shared pointers to Player objects.

3.12.2.2 getGeneralActions()

```
const std::vector< std::shared_ptr< Action > > & Game::getGeneralActions ( ) const
```

Gets the general actions available in the game.

Returns

Const reference to a vector of shared pointers to Action objects.

3.12.2.3 getState()

```
bool Game::getState ( ) const
```

Gets the current state indicating if the game is running.

Returns

True if the game is running, false otherwise.

3.12.2.4 init_actions()

```
void Game::init_actions (
            std::shared_ptr< Player > bard,
            std::shared_ptr< Player > druid,
            std::shared_ptr< Player > knight,
            std::shared_ptr< Player > wizard )
```

Initializes actions available to player classes and general actions.

Parameters

bard	Bard player instance.
druid	Druid player instance.
knight	Knight player instance.
wizard	Wizard player instance.

3.12.2.5 init_items()

```
void Game::init_items (
            std::shared_ptr< Player > bard,
            std::shared_ptr< Player > druid,
            std::shared_ptr< Player > knight,
            std::shared_ptr< Player > wizard )
```

Initializes items associated with player classes and enemies.

Parameters

bard	Bard player instance.
druid	Druid player instance.
knight	Knight player instance.
wizard	Wizard player instance.

3.12.2.6 is_numeric()

```
bool Game::is_numeric (
            std::string & input )
```

Checks if a string input is numeric.

Parameters

input	String input.

Returns

True if numeric, false otherwise.

The documentation for this class was generated from the following files:

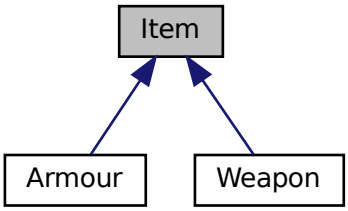
- source/Game.h
- source/Game.cpp

3.13 Item Class Reference

Represents an item that characters can equip or use, with various stat bonuses.

```
#include <Item.h>
```

Inheritance diagram for Item:



Public Member Functions

- Item (const std::string &name, const std::string &desc, int durability, int type, int health_bonus, int defence_bonus, int mana_bonus, int attack_bonus, double luck_bonus, int hb_bonus, double crit_chance_bonus, int int_bonus)

  Constructs an Item with specified attributes.
- virtual ~Item ()

  Destructor.
- void setName (const std::string &n)

  Sets the name of the item.
- const std::string & getName () const

  Gets the name of the item.
- void setDesc (const std::string &desc)

  Sets the description of the item.
- const std::string & getDesc () const

  Gets the description of the item.
- void setDurability (int d)

  Sets the durability of the item.
- int getDurability () const

  Gets the durability of the item.
- void setHealthBonus (int h)

  Sets health bonus.
- int getHealthBonus () const

  Gets health bonus.
- void setDefenceBonus (int d)

  Sets defense bonus.
- int getDefenceBonus () const

  Gets defense bonus.
- void setManaBonus (int m)

  Sets mana bonus.
- int getManaBonus () const

  Gets mana bonus.
- void setAttackBonus (int a)

  Sets attack bonus.
- int getAttackBonus () const

Gets attack bonus.

- void setLuckBonus (double l)

  Sets luck bonus.
- double getLuckBonus () const

  Gets luck bonus.
- void setHBBonus (int hbb)

  Sets healing bonus.
- int getHBBonus () const

  Gets healing bonus.
- void setCritChanceBonus (double c)

  Sets critical chance bonus.
- int getCritChanceBonus () const

  Gets critical chance bonus.
- void setIntBonus (int i)

  Sets intelligence bonus.
- int getIntBonus () const

  Gets intelligence bonus.
- int getType () const

  Gets the item type code.

Protected Attributes

- std::string name

  Item name.
- std::string description

  Item description.
- int durability

  Remaining durability.
- int type

  Item type code.
- int health_bonus

  Bonus to health.
- int defence_bonus

  Bonus to defense.
- int mana_bonus

  Bonus to mana.
- int attack_bonus

  Bonus to attack power.
- double luck_bonus

  Bonus to luck.
- int hb_bonus

  Healing bonus.
- double crit_chance_bonus

  Critical hit chance bonus.
- int int_bonus

  Intelligence bonus.

3.13.1 Detailed Description

Represents an item that characters can equip or use, with various stat bonuses.

3.13.2 Constructor & Destructor Documentation

3.13.2.1 Item()

```
Item::Item (
            const std::string & name,
            const std::string & desc,
            int durability,
            int type,
            int health_bonus,
            int defence_bonus,
            int mana_bonus,
            int attack_bonus,
            double luck_bonus,
            int hb_bonus,
            double crit_chance_bonus,
            int int_bonus )
```

Constructs an Item with specified attributes.

Parameters

name	Item name.
desc	Item description.
durability	Durability of the item.
type	Item type code.
health_bonus	Health bonus given by the item.
defence_bonus	Defense bonus.
mana_bonus	Mana bonus.
attack_bonus	Attack bonus.
luck_bonus	Luck bonus.
hb_bonus	Healing bonus.
crit_chance_bonus	Critical chance bonus.
int_bonus	Intelligence bonus.

3.13.3 Member Function Documentation

3.13.3.1 getAttackBonus()

```
int Item::getAttackBonus ( ) const
```

Gets attack bonus.

Returns

Attack bonus value.

3.13.3.2 getCritChanceBonus()

```
int Item::getCritChanceBonus ( ) const
```

Gets critical chance bonus.

Returns

Critical chance bonus value.

3.13.3.3 getDefenceBonus()

```
int Item::getDefenceBonus ( ) const
```

Gets defense bonus.

Returns

Defense bonus value.

3.13.3.4 getDesc()

```
const std::string & Item::getDesc ( ) const
```

Gets the description of the item.

Returns

Const reference to description string.

3.13.3.5 getDurability()

```
int Item::getDurability ( ) const
```

Gets the durability of the item.

Returns

Durability value.

3.13.3.6 getHBBonus()

```
int Item::getHBBonus ( ) const
```

Gets healing bonus.

Returns

Healing bonus value.

3.13.3.7 getHealthBonus()

```
int Item::getHealthBonus ( ) const
```

Gets health bonus.

Returns

Health bonus value.

3.13.3.8 getIntBonus()

```
int Item::getIntBonus ( ) const
```

Gets intelligence bonus.

Returns

Intelligence bonus value.

3.13.3.9 getLuckBonus()

```
double Item::getLuckBonus ( ) const
```

Gets luck bonus.

Returns

Luck bonus value.

3.13.3.10 getManaBonus()

```
int Item::getManaBonus ( ) const
```

Gets mana bonus.

Returns

Mana bonus value.

3.13.3.11 getName()

```
const std::string & Item::getName ( ) const
```

Gets the name of the item.

Returns

Const reference to name string.

3.13.3.12 getType()

```
int Item::getType ( ) const
```

Gets the item type code.

Returns

Item type integer.

3.13.3.13 setAttackBonus()

```
void Item::setAttackBonus (
            int a )
```

Sets attack bonus.

Parameters

a	Attack bonus value.

3.13.3.14 setCritChanceBonus()

```
void Item::setCritChanceBonus (
            double c )
```

Sets critical chance bonus.

Parameters

c	Critical chance bonus value.

3.13.3.15 setDefenceBonus()

```
void Item::setDefenceBonus (
            int d )
```

Sets defense bonus.

Parameters

d	Defense bonus value.

3.13.3.16 setDesc()

```
void Item::setDesc (
            const std::string & desc )
```

Sets the description of the item.

Parameters

desc	New description.

3.13.3.17 setDurability()

```
void Item::setDurability (
            int d )
```

Sets the durability of the item.

Parameters

d	Durability value.

3.13.3.18 setHBBonus()

```
void Item::setHBBonus (
            int hbb )
```

Sets healing bonus.

Parameters

hbb	Healing bonus value.

3.13.3.19 setHealthBonus()

```
void Item::setHealthBonus (
            int h )
```

Sets health bonus.

Parameters

h	Health bonus value.

3.13.3.20 setIntBonus()

```
void Item::setIntBonus (
            int i )
```

Sets intelligence bonus.

Parameters

i	Intelligence bonus value.

3.13.3.21 setLuckBonus()

```
void Item::setLuckBonus (
            double l )
```

Sets luck bonus.

Parameters

l	Luck bonus value.

3.13.3.22 setManaBonus()

```
void Item::setManaBonus (
            int m )
```

Sets mana bonus.

Parameters

m	Mana bonus value.

3.13.3.23 setName()

```
void Item::setName (
            const std::string & n )
```

Sets the name of the item.

Parameters

n	New name.

The documentation for this class was generated from the following files:

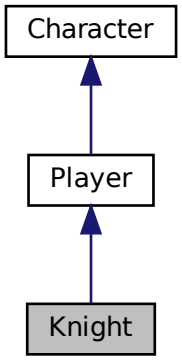
- source/Item.h
- source/Item.cpp

3.14 Knight Class Reference

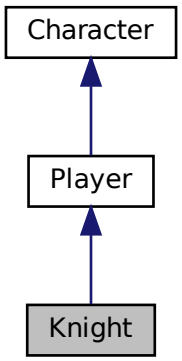
Player class representing a Knight character with critical chance attribute.

```
#include <Knight.h>
```

Inheritance diagram for Knight:



Collaboration diagram for Knight:



Public Member Functions

- Knight (const std::string &name, int health, int mana, int attack_power, int defence, int hr, const std::string &desc, std::shared_ptr< Item > item, double crit_chance)

  Constructs a Knight with given parameters.
- Knight ()

  Default constructor for Knight with initial stats.
- ~Knight ()

  Destructor.
- void setCritChance (double chance) override

  Sets the critical chance.

- double getCritChance () const override

  Gets the critical chance.
- std::shared_ptr< Player > clone () const override

  Clones the Knight object.

Protected Attributes

- double crit_chance

  Critical hit chance.

3.14.1 Detailed Description

Player class representing a Knight character with critical chance attribute.

Inherits from Player and adds critical chance functionality.

3.14.2 Constructor & Destructor Documentation

3.14.2.1 Knight()

```
Knight::Knight (
            const std::string & name,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            const std::string & desc,
            std::shared_ptr< Item > item,
            double crit_chance )
```

Constructs a Knight with given parameters.

Parameters

name	Character name.
health	Health points.
mana	Mana points.
attack_power	Attack power.
defence	Defense rating.
hr	Healing rate.
desc	Description.
item	Starting item.
crit_chance	Critical hit chance.

3.14.3 Member Function Documentation

3.14.3.1 clone()

```
std::shared_ptr< Player > Knight::clone ( ) const  [override], [virtual]
```

Clones the Knight object.

Returns

Shared pointer to a copy of this Knight.

Reimplemented from Player.

3.14.3.2 getCritChance()

```
double Knight::getCritChance ( ) const  [override], [virtual]
```

Gets the critical chance.

Returns

Critical hit chance value.

Reimplemented from Character.

3.14.3.3 setCritChance()

```
void Knight::setCritChance (
            double chance )  [override], [virtual]
```

Sets the critical chance.

Parameters

chance	Critical hit chance value.

Reimplemented from Character.

The documentation for this class was generated from the following files:

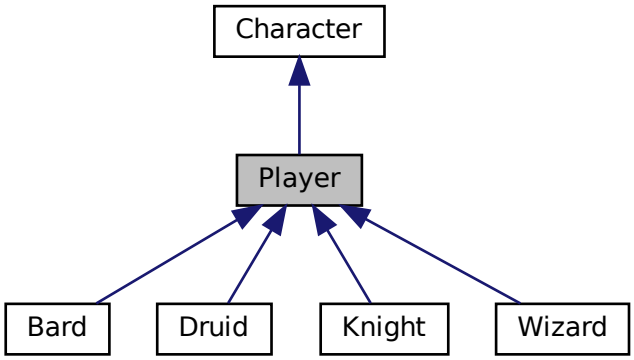
- source/Knight.h
- source/Knight.cpp

3.15 Player Class Reference

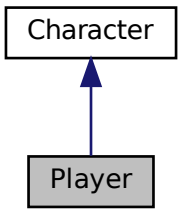
Represents a player character, inheriting from Character.

`#include <Player.h>`

Inheritance diagram for Player:



Collaboration diagram for Player:



Public Member Functions

- Player ()

  Default constructor.
- Player (const std::string &name, int cid, int health, int mana, int attack_power, int defence, int hr, const std::string &desc, std::shared_ptr< Item > item)

  Constructs a player with customized parameters.
- ~Player ()

  Destructor.
- void setItem (std::shared_ptr< Item > i) override

Sets an equipped item.

- std::shared_ptr< Item > getItem () const override

  Gets the equipped item.
- void setDesc (const std::string &desc)

  Sets the class description.
- std::string getDesc ()

  Returns the class description.
- void setClassItems (const std::vector< std::shared_ptr< Item >> &v)

  Sets the class-specific items available.
- const std::vector< std::shared_ptr< Item > > & getClassItems ()

  Returns class-specific items.
- void setClassActions (const std::vector< std::shared_ptr< Action >> &v)

  Sets the class-specific actions available.
- const std::vector< std::shared_ptr< Action > > & getClassActions ()

  Returns class-specific actions.
- virtual std::shared_ptr< Player > clone () const

  Clones the player object.

Protected Attributes

- std::vector< std::shared_ptr< Item > > class_items

  Items specific to this player class.
- std::vector< std::shared_ptr< Action > > class_actions

  Actions specific to this player class.
- std::shared_ptr< Item > item

  Currently equipped item.
- std::string description

  Class description.

3.15.1 Detailed Description

Represents a player character, inheriting from Character.

Contains inventory items and actions specific to player subclasses.

3.15.2 Constructor & Destructor Documentation

3.15.2.1 Player()

```
Player::Player (
            const std::string & name,
            int cid,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            const std::string & desc,
            std::shared_ptr< Item > item )
```

Constructs a player with customized parameters.

Parameters

name	Character name.
cid	Class ID.
health	Health points.
mana	Mana points.
attack_power	Attack power.
defence	Defense rating.
hr	Healing rate.
desc	Description text.
item	Starting item.

3.15.3 Member Function Documentation

3.15.3.1 clone()

```
std::shared_ptr< Player > Player::clone ( ) const  [virtual]
```

Clones the player object.

Returns

Shared pointer to a copy of this Player.

Reimplemented in Wizard, Knight, Druid, and Bard.

3.15.3.2 getClassActions()

```
const std::vector< std::shared_ptr< Action > > & Player::getClassActions ( )
```

Returns class-specific actions.

Returns

Vector of action pointers.

3.15.3.3 getClassItems()

```
const std::vector< std::shared_ptr< Item > > & Player::getClassItems ( )
```

Returns class-specific items.

Returns

Vector of item pointers.

3.15.3.4 getDesc()

```
std::string Player::getDesc ( )
```

Returns the class description.

Returns

Description string.

3.15.3.5 getItem()

```
std::shared_ptr< Item > Player::getItem ( ) const  [override], [virtual]
```

Gets the equipped item.

Returns

Pointer to equipped item.

Reimplemented from Character.

3.15.3.6 setClassActions()

```
void Player::setClassActions (
            const std::vector< std::shared_ptr< Action >> & v )
```

Sets the class-specific actions available.

Parameters

v	Vector of action pointers.

3.15.3.7 setClassItems()

```
void Player::setClassItems (
            const std::vector< std::shared_ptr< Item >> & v )
```

Sets the class-specific items available.

Parameters

v	Vector of item pointers.

3.15.3.8 setDesc()

```
void Player::setDesc (
            const std::string & desc )
```

Sets the class description.

Parameters

desc	Description string.

3.15.3.9 setItem()

```
void Player::setItem (
            std::shared_ptr< Item > i )  [override], [virtual]
```

Sets an equipped item.

Parameters

i	Item to equip.

Reimplemented from Character.

The documentation for this class was generated from the following files:

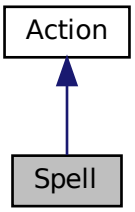
- source/Player.h
- source/Player.cpp

3.16 Spell Class Reference

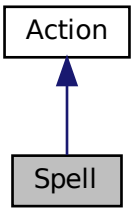
Represents a spell action with effect based on intelligence and attack power.

`#include <Spell.h>`

Inheritance diagram for Spell:



Collaboration diagram for Spell:



Public Member Functions

- Spell (const std::string &name, const std::string &desc, int mana)

  Constructs a Spell action.
- void exucute (Character &performer, Character &target, Battle &battle, std::shared_ptr< Item > item, Character &item_target)

  Executes the spell's effect targeting a character within a battle.

Additional Inherited Members

3.16.1 Detailed Description

Represents a spell action with effect based on intelligence and attack power.

Derived from Action.

3.16.2 Constructor & Destructor Documentation

3.16.2.1 Spell()

```
Spell::Spell (
            const std::string & name,
            const std::string & desc,
            int mana )
```

Constructs a Spell action.

Parameters

name	Name of the spell.
desc	Description of the spell.
mana	Mana cost for casting.

3.16.3 Member Function Documentation

3.16.3.1 exucute()

```
void Spell::exucute (
            Character & performer,
            Character & target,
            Battle & battle,
            std::shared_ptr< Item > item,
            Character & item_target )
```

Executes the spell's effect targeting a character within a battle.

Parameters

performer	Character casting the spell.
target	Target character.
battle	Current battle context.
item	Optional item involved in spell effect.
item_target	Character receiving the item effect.

The documentation for this class was generated from the following files:

- source/Spell.h
- source/Spell.cpp

3.17 Statistics Class Reference

Keeps track of the player's score and number of games played.

`#include <Statistics.h>`

Public Member Functions

- Statistics ()
  Default constructor initializes score and games played.
- void set_score (int score)
  Sets the player score.
- int get_score ()
  Gets the player score.
- void set_games (int games)
  Sets the number of games played.
- int get_games ()
  Gets the number of games played.

3.17.1 Detailed Description

Keeps track of the player's score and number of games played.

3.17.2 Member Function Documentation

3.17.2.1 get_games()

```
int Statistics::get_games ( )
```

Gets the number of games played.

Returns

Number of games played.

3.17.2.2 get_score()

```
int Statistics::get_score ( )
```

Gets the player score.

Returns

Current player score.

3.17.2.3 set_games()

```
void Statistics::set_games (
            int games )
```

Sets the number of games played.

Parameters

games	Number of games played.

3.17.2.4 set_score()

```
void Statistics::set_score (
            int score )
```

Sets the player score.

Parameters

score	New player score.

The documentation for this class was generated from the following files:

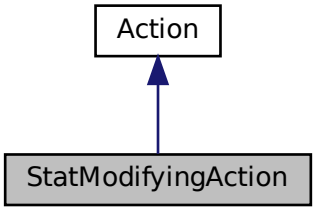
- source/Statistics.h
- source/Statistics.cpp

3.18 StatModifyingAction Class Reference

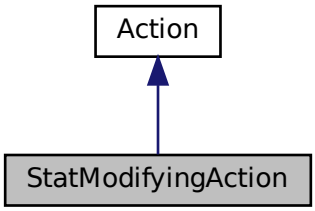
Action that modifies character stats such as health, mana, attack, or defense.

```
#include <StatModifyingAction.h>
```

Inheritance diagram for StatModifyingAction:



Collaboration diagram for StatModifyingAction:



Public Member Functions

- StatModifyingAction (const std::string &name, const std::string &desc, int mana, int modified_stat, int quantity)

  Constructs a StatModifyingAction.
- void setModifiedStat (int stat)

  Sets the stat to be modified.
- int getModifiedStat () const

  Gets the stat to be modified.
- void setQuantity (int quantity)

  Sets the quantity for stat modification.
- int getQuantity () const

  Gets the quantity of stat modification.
- void execute (Character &performer, Character &target, Battle &battle, std::shared_ptr< Item >, Character &) override

  Executes the stat modification action on the target character.

Protected Attributes

- int modified_stat

  Stat identifier to modify.
- int quantity

  Amount to modify the stat.

3.18.1 Detailed Description

Action that modifies character stats such as health, mana, attack, or defense.

3.18.2 Constructor & Destructor Documentation

3.18.2.1 StatModifyingAction()

```
StatModifyingAction::StatModifyingAction (
            const std::string & name,
            const std::string & desc,
            int mana,
            int modified_stat,
            int quantity )
```

Constructs a StatModifyingAction.

Parameters

name	Name of the action.
desc	Description of the action.
mana	Mana cost.
modified_stat	Stat identifier to modify.
quantity	Quantity to modify stat by (positive or negative).

3.18.3 Member Function Documentation

3.18.3.1 execute()

```
void StatModifyingAction::execute (
            Character & performer,
            Character & target,
            Battle & battle,
            std::shared_ptr< Item > ,
            Character &  )  [override], [virtual]
```

Executes the stat modification action on the target character.

Parameters

performer	Character performing the action.
target	Character targeted.
battle	Current battle context.
item	Optional item used.
item_target	Character affected by item.

Reimplemented from Action.

3.18.3.2 getModifiedStat()

```
int StatModifyingAction::getModifiedStat ( ) const
```

Gets the stat to be modified.

Returns

Stat ID.

3.18.3.3 getQuantity()

```
int StatModifyingAction::getQuantity ( ) const
```

Gets the quantity of stat modification.

Returns

Amount of modification.

3.18.3.4 setModifiedStat()

```
void StatModifyingAction::setModifiedStat (
            int stat )
```

Sets the stat to be modified.

Parameters

stat	Stat ID.

3.18.3.5 setQuantity()

```
void StatModifyingAction::setQuantity (
            int quantity )
```

Sets the quantity for stat modification.

Parameters

quantity	Amount to modify.

The documentation for this class was generated from the following files:

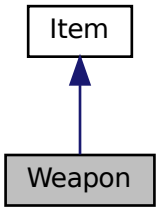
- source/StatModifyingAction.h
- source/StatModifyingAction.cpp

3.19 Weapon Class Reference

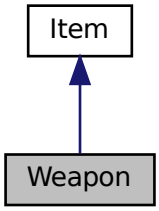
Represents a weapon item with additional level attribute.

`#include <Weapon.h>`

Inheritance diagram for Weapon:



Collaboration diagram for Weapon:



Public Member Functions

- Weapon (const std::string &name, const std::string &desc, int durability, int defence_bonus, int attack_bonus, double crit_chance_bonus, int lvl)

  Constructs a Weapon with specified attributes.
- void setLevel (int lvl)

  Sets the weapon's level.
- int getLevel () const

  Gets the weapon's level.

Protected Attributes

- int level

  Weapon level.

3.19.1 Detailed Description

Represents a weapon item with additional level attribute.

Inherits from Item.

3.19.2 Constructor & Destructor Documentation

3.19.2.1 Weapon()

```
Weapon::Weapon (
            const std::string & name,
            const std::string & desc,
            int durability,
            int defence_bonus,
            int attack_bonus,
            double crit_chance_bonus,
            int lvl )
```

Constructs a Weapon with specified attributes.

Parameters

name	Weapon name.
desc	Weapon description.
durability	Durability value.
defence_bonus	Defense bonus.
attack_bonus	Attack bonus.
crit_chance_bonus	Critical chance bonus.
lvl	Weapon level.

3.19.3 Member Function Documentation

3.19.3.1 getLevel()

```
int Weapon::getLevel ( ) const
```

Gets the weapon's level.

Returns

Weapon level.

3.19.3.2 setLevel()

```
void Weapon::setLevel (
            int lvl )
```

Sets the weapon's level.

Parameters

lvl	Level to set.

The documentation for this class was generated from the following files:

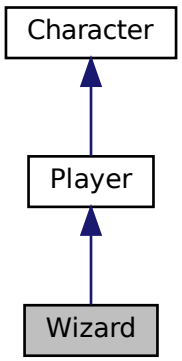
- source/Weapon.h
- source/Weapon.cpp

3.20 Wizard Class Reference

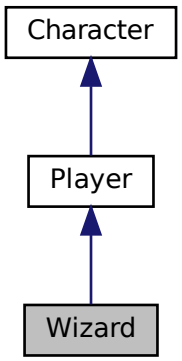
Player class representing a Wizard with intelligence attribute.

```
#include <Wizard.h>
```

Inheritance diagram for Wizard:



Collaboration diagram for Wizard:



Public Member Functions

- Wizard (const std::string &name, int health, int mana, int attack_power, int defence, int hr, const std::string &desc, std::shared_ptr< Item > item, int intelligence)

  Constructs a Wizard with specific stats.
- Wizard ()

  Default constructor initializing base Wizard stats.
- ~Wizard ()

  Destructor.
- void setIntelligence (int intel) override

  Sets the intelligence attribute.
- int getIntelligence () const override

  Gets the intelligence attribute.
- std::shared_ptr< Player > clone () const override

  Clones the Wizard object.

Protected Attributes

- int intelligence

  Intelligence stat.

3.20.1 Detailed Description

Player class representing a Wizard with intelligence attribute.

Inherits from Player and adds intelligence functionality.

3.20.2 Constructor & Destructor Documentation

3.20.2.1 Wizard()

```
Wizard::Wizard (
            const std::string & name,
            int health,
            int mana,
            int attack_power,
            int defence,
            int hr,
            const std::string & desc,
            std::shared_ptr< Item > item,
            int intelligence )
```

Constructs a Wizard with specific stats.

Parameters

name	Character name.
health	Health points.
mana	Mana points.
attack_power	Attack value.
defence	Defense value.
hr	Healing rate.
desc	Description string.
item	Starting item.
intelligence	Intelligence stat.

3.20.3 Member Function Documentation

3.20.3.1 clone()

```
std::shared_ptr< Player > Wizard::clone ( ) const  [override], [virtual]
```

Clones the Wizard object.

Returns

Shared pointer to a copied Wizard.

Reimplemented from Player.

3.20.3.2 getIntelligence()

```
int Wizard::getIntelligence ( ) const  [override], [virtual]
```

Gets the intelligence attribute.

Returns

Intelligence value.

Reimplemented from Character.

3.20.3.3 setIntelligence()

```
void Wizard::setIntelligence (
            int intel )  [override], [virtual]
```

Sets the intelligence attribute.

Parameters

intel	New intelligence value.

Reimplemented from Character.

The documentation for this class was generated from the following files:

- source/Wizard.h
- source/Wizard.cpp

Index

Action, 5
    Action, 6
    attempt, 6
    execute, 7
    getDesc, 7
    getManaUsage, 7
    getName, 8
    setDesc, 8
    setManaUsage, 8
    setName, 9
Armour, 9
    Armour, 10
    getWeight, 11
    setWeight, 11
attempt
    Action, 6
    Dodge, 33

Bard, 11
    Bard, 13
    clone, 14
    getLuck, 14
    setLuck, 14
Battle, 15
    calculate_damage, 16
    get_turn, 16
    get_turn_block, 16
    getActionsCatalogue, 16
    getItemsCatalogue, 17
    getReportMessage, 17
    is_critical, 17
    is_enough_mana, 18
    set_turn, 18
    set_turn_block, 18
    setActionsCatalogue, 19
    setItemsCatalogue, 19
    setReportMessage, 19

calculate_damage
    Battle, 16
Character, 19
    Character, 22
    getArmour, 22
    getAttackPower, 22
    getCid, 23
    getCritChance, 23
    getDefence, 23
    getHealingBonus, 23
    getHealingRate, 24
    getHealth, 24
    getIntelligence, 24
    getItem, 24
    getLuck, 25
    getMana, 25
    getName, 25
    getWeapon, 25
    setArmour, 26
    setAttackPower, 26
    setCritChance, 26
    setDefence, 27
    setHealingBonus, 27
    setHealingRate, 27
    setHealth, 27
    setIntelligence, 28
    setItem, 28
    setLuck, 28
    setMana, 29
    setName, 29
    setWeapon, 29
chooseAction
    EnemyAI, 42
clone
    Bard, 14
    Druid, 36
    Knight, 60
    Player, 63
    Wizard, 77

Distract, 30
    Distract, 31
    execute, 31
Dodge, 32
    attempt, 33
    Dodge, 33
Druid, 34
    clone, 36
    Druid, 35
    getHealingBonus, 36
    setHealingBonus, 36

Enemy, 37
    Enemy, 38
    getCritChance, 39
    getEquippingFlag, 39
    getNextAction, 39
    setCritChance, 40
    setEquippingFlag, 41
    setNextAction, 41
EnemyAI, 41
    chooseAction, 42

Equip, 42
    Equip, 43
    execute, 44
execute
    Action, 7
    Distract, 31
    Equip, 44
    StatModifyingAction, 71
exucute
    Spell, 67

Game, 44
    getCharacters, 46
    getGeneralActions, 47
    getState, 47
    init_actions, 47
    init_items, 48
    is_numeric, 48
get_games
    Statistics, 68
get_score
    Statistics, 68
get_turn
    Battle, 16
get_turn_block
    Battle, 16
getActionsCatalogue
    Battle, 16
getArmour
    Character, 22
getAttackBonus
    Item, 51
getAttackPower
    Character, 22
getCharacters
    Game, 46
getCid
    Character, 23
getClassActions
    Player, 63
getClassItems
    Player, 63
getCritChance
    Character, 23
    Enemy, 39
    Knight, 60
getCritChanceBonus
    Item, 51
getDefence
    Character, 23
getDefenceBonus
    Item, 52
getDesc
    Action, 7
    Item, 52
    Player, 64
getDurability
    Item, 52
getEquippingFlag
    Enemy, 39
getGeneralActions
    Game, 47
getHBBonus
    Item, 52
getHealingBonus
    Character, 23
    Druid, 36
getHealingRate
    Character, 24
getHealth
    Character, 24
getHealthBonus
    Item, 53
getIntBonus
    Item, 53
getIntelligence
    Character, 24
    Wizard, 77
getItem
    Character, 24
    Player, 64
getItemsCatalogue
    Battle, 17
getLevel
    Weapon, 74
getLuck
    Bard, 14
    Character, 25
getLuckBonus
    Item, 53
getMana
    Character, 25
getManaBonus
    Item, 53
getManaUsage
    Action, 7
getModifiedStat
    StatModifyingAction, 71
getName
    Action, 8
    Character, 25
    Item, 54
getNextAction
    Enemy, 39
getQuantity
    StatModifyingAction, 72
getReportMessage
    Battle, 17
getState
    Game, 47
getType
    Item, 54
getWeapon
    Character, 25
getWeight
    Armour, 11

init_actions

Game, 47
init_items
Game, 48
is_critical
Battle, 17
is_enough_mana
Battle, 18
is_numeric
Game, 48
Item, 48
getAttackBonus, 51
getCritChanceBonus, 51
getDefenceBonus, 52
getDesc, 52
getDurability, 52
getHBBonus, 52
getHealthBonus, 53
getIntBonus, 53
getLuckBonus, 53
getManaBonus, 53
getName, 54
getType, 54
Item, 51
setAttackBonus, 54
setCritChanceBonus, 54
setDefenceBonus, 55
setDesc, 55
setDurability, 55
setHBBonus, 56
setHealthBonus, 56
setIntBonus, 56
setLuckBonus, 56
setManaBonus, 57
setName, 57

Knight, 57
clone, 60
getCritChance, 60
Knight, 59
setCritChance, 60

Player, 61
clone, 63
getClassActions, 63
getClassItems, 63
getDesc, 64
getItem, 64
Player, 62
setClassActions, 64
setClassItems, 65
setDesc, 65
setItem, 65

set_games
Statistics, 68
set_score
Statistics, 69
set_turn
Battle, 18
set_turn_block
Battle, 18
setActionsCatalogue
Battle, 19
setArmour
Character, 26
setAttackBonus
Item, 54
setAttackPower
Character, 26
setClassActions
Player, 64
setClassItems
Player, 65
setCritChance
Character, 26
Enemy, 40
Knight, 60
setCritChanceBonus
Item, 54
setDefence
Character, 27
setDefenceBonus
Item, 55
setDesc
Action, 8
Item, 55
Player, 65
setDurability
Item, 55
setEquippingFlag
Enemy, 41
setHBBonus
Item, 56
setHealingBonus
Character, 27
Druid, 36
setHealingRate
Character, 27
setHealth
Character, 27
setHealthBonus
Item, 56
setIntBonus
Item, 56
setIntelligence
Character, 28
Wizard, 78
setItem
Character, 28
Player, 65
setItemsCatalogue
Battle, 19
setLevel
Weapon, 74
setLuck
Bard, 14
Character, 28

setLuckBonus
Item, 56
setMana
Character, 29
setManaBonus
Item, 57
setManaUsage
Action, 8
setModifiedStat
StatModifyingAction, 72
setName
Action, 9
Character, 29
Item, 57
setNextAction
Enemy, 41
setQuantity
StatModifyingAction, 72
setReportMessage
Battle, 19
setWeapon
Character, 29
setWeight
Armour, 11
Spell, 66
exucute, 67
Spell, 67
Statistics, 68
get_games, 68
get_score, 68
set_games, 68
set_score, 69
StatModifyingAction, 69
execute, 71
getModifiedStat, 71
getQuantity, 72
setModifiedStat, 72
setQuantity, 72
StatModifyingAction, 70

Weapon, 73
getLevel, 74
setLevel, 74
Weapon, 74
Wizard, 75
clone, 77
getIntelligence, 77
setIntelligence, 78
Wizard, 76